

Laboratorio di Metodi numerici per il calcolo tensoriale

Lezione 1

Tensori in Matlab: funzionalità di base

- (i) Per definire un tensore in Matlab/Octave si può usare la stessa sintassi utilizzata per le matrici, aggiungendo gli indici necessari. Ad esempio il comando `randn(5,6,7,8)` definisce un tensore $\mathbb{R}^{5 \times 6 \times 7 \times 8}$ con entrate generate casualmente da una distribuzione normale.
- (ii) Le vettorizzazioni, matricizzazioni, tensorizzazioni, che mantengono l'ordinamento naturale delle entrate di una struttura dati possono essere ottenute con il comando `reshape` che ha la stessa funzione dell'operatore `reshape` visto a lezione. La vettorizzazione di un tensore/matrice `T` può essere ottenuta anche con il comando `T(:)`.
- (iii) Le funzioni `sub2ind` e `ind2sub` corrispondono alle funzioni di ordinamento naturale e alla sua inversa ($\mathbb{L}$ e $\mathbb{L}^{-1}$, nella notazione delle lezioni).
- (iv) La funzione `permute` effettua una permutazione dei modi del tensore secondo una permutazione data in input. Ad esempio le 4 matricizzazioni di `T=randn(2,3,4,5)` sono ottenute con i comandi

```
T1 = reshape(T, 2, 3 * 4 * 5);  
S = permute(T, [2 1 3 4]);  
T2 = reshape(S, 3, 2 * 4 * 5);  
S = permute(T, [3 1 2 4]);  
T3 = reshape(S, 4, 2 * 3 * 5);  
S = permute(T, [4 1 2 3]);  
T4 = reshape(S, 5, 2 * 3 * 4);
```

Moltiplicazioni tensore matrice

Esercizio 1. Si implementino 2 funzioni

```
function T = modemul_slow(X, mode, U)
function T = modemul_fast(X, mode, U)
```

per la valutazione del prodotto tensore matrice $X \times_{\text{mode}} U$, per tensori 3D ($\text{mode} \in \{1, 2, 3\}$). La prima funzione forma esplicitamente le matricizzazioni utilizzando la funzione `permute`, mentre la seconda evita le permutazioni con le tecniche viste a lezione. Si eseguano le due funzioni sui modi 2 e 3 per tensori $n \times n \times n$ e $U \in \mathbb{R}^{n \times n}$ (casuale) per vari n . In particolare, sia per il modo 2 che per il modo 3, si generi un grafico per apprezzare il confronto dei tempi di calcolo dei due approcci al crescere di n .

Esercizio 2 (facoltativo). Si estendano le due funzioni, ed il relativo confronto, dell'esercizio precedente al caso di tensori d -dimensionali.

Tensori che rappresentano il prodotto di matrici

Indichiamo con $\langle m, n, p \rangle$ la funzione bilineare che rappresenta la moltiplicazione fra una matrice $U \in \mathbb{R}^{m \times n}$ con una $V \in \mathbb{R}^{n \times p}$. Assumendo l'ordinamento naturale sulle entrate di U e V e l'ordinamento inverso su quello di $W = UV$ si ottiene una rappresentazione $\mathcal{X} \in \mathbb{R}^{mn \times np \times mp}$ di $\langle m, n, p \rangle$ tale che

$$\text{vec}(W^T) = \mathcal{X} \times_1 \text{vec}(U)^T \times_2 \text{vec}(V)^T. \quad (1)$$

In particolare il tensore $\mathcal{X}$ ha entrate uguali a 1 (e le restanti zero) in corrispondenza degli indici (i, j, k) tali che

$$(\mathbb{L}^*)^{-1}(k; m \times p) = (k_r, k_c), \quad \mathbb{L}^{-1}(i; m \times n) = (k_r, h), \quad \mathbb{L}^{-1}(j; n \times p) = (h, k_c),$$

per $h = 1, \dots, p$ (capire perchè). Ad esempio, nel caso $m = n = p = 2$, si ottiene il tensore con fette frontali

$$X_1 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad X_2 = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad X_3 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}, \quad X_4 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Esercizio 3. Si scriva una funzione

```
function X = matmul(m,n,p)
```

che dati i parametri m, n, p , restituisce il tensore $\mathcal{X}$ che rappresenta $\langle m, n, p \rangle$. Si testi la correttezza del tensore calcolato valutando (1) e confrontandola con il risultato calcolato in maniera usuale, per valori diversi di m, n, p e matrici U, V generate casualmente.

Se si ha a disposizione una fattorizzazione CP di rango r $\mathcal{X} = \llbracket A, B, C \rrbracket$, possiamo riscrivere (1) come:

$$\text{vec}(W^T) = \sum_{\ell=1}^r (a_\ell^T \text{vec}(U)) \cdot (b_\ell^T \text{vec}(V)) \cdot c_\ell \quad (2)$$

dove a_ℓ, b_ℓ, c_ℓ indicano le colonne di A, B, C .

Esercizio 4. Si scriva una funzione

```
function W = matmul_cp(U, V, A, B, C)
```

che calcola $W = U \cdot V$, utilizzando (2). Si testi la correttezza dell'implementazione sul prodotto di matrici 2×2 usando la fattorizzazione CP di Strassen

```
A = [1 0 1 0 1 -1 0;
      0 1 0 0 0 1 0;
      0 0 0 0 1 0 1;
      1 1 0 1 0 0 -1];
B = [1 1 0 -1 0 1 0;
      0 0 0 1 0 0 1;
      0 0 1 0 0 1 0;
      1 0 -1 0 1 0 1];
C = [1 0 0 1 -1 0 1;
      0 0 1 0 1 0 0;
      0 1 0 1 0 0 0;
      1 -1 1 0 0 1 0];
```

su matrici 2×2 casuali.

Prodotto veloce tra matrici

Date due matrici $U \in \mathbb{R}^{m \times n}$ e $V \in \mathbb{R}^{n \times p}$, con $m, n, p > 2$ possiamo sempre considerarle partizionate a blocchi come:

$$\begin{bmatrix} U_{11} & U_{12} \\ U_{21} & U_{22} \end{bmatrix}, \quad V = \begin{bmatrix} V_{11} & V_{12} \\ V_{21} & V_{22} \end{bmatrix}$$

dove $U_{11} \in \mathbb{R}^{\lfloor \frac{m}{2} \rfloor \times \lfloor \frac{n}{2} \rfloor}$, $V_{11} \in \mathbb{R}^{\lfloor \frac{n}{2} \rfloor \times \lfloor \frac{p}{2} \rfloor}$ e $U_{22} \in \mathbb{R}^{\lceil \frac{m}{2} \rceil \times \lceil \frac{n}{2} \rceil}$, $V_{22} \in \mathbb{R}^{\lceil \frac{n}{2} \rceil \times \lceil \frac{p}{2} \rceil}$. In particolare, il prodotto UV possiamo vederlo come un prodotto fra matrici 2×2 a blocchi

$$U \cdot V = \begin{bmatrix} U_{11}V_{11} + U_{12}V_{21} & U_{11}V_{12} + U_{12}V_{22} \\ U_{21}V_{11} + U_{22}V_{21} & U_{21}V_{12} + U_{22}V_{22} \end{bmatrix}$$

ed utilizzare l'algoritmo di Strassen per valutarlo (al posto del classico righe $\times$ colonne scritto qua sopra). Questo passaggio richiede una precisazione: quando si usa la formula (2) in un contesto in cui le entrate di U e V non sono scalari (ma matrici) si intende che:

- $\text{vec}(U) = \begin{bmatrix} U_{11} \\ U_{21} \\ U_{12} \\ U_{22} \end{bmatrix}$, $\text{vec}(V) = \begin{bmatrix} V_{11} \\ V_{21} \\ V_{12} \\ V_{22} \end{bmatrix}$, $\text{vec}(W^T) = \begin{bmatrix} W_{11} \\ W_{12} \\ W_{21} \\ W_{22} \end{bmatrix}$,
- $a_\ell^T \text{vec}(U) = \sum_{i=1}^4 (a_\ell)_i \text{vec}(U)_i$, $b_\ell^T \text{vec}(V) = \sum_{i=1}^4 (b_\ell)_i \text{vec}(V)_i$,

$$\bullet \quad M \cdot c_\ell = \begin{bmatrix} (c_\ell)_1 M \\ (c_\ell)_2 M \\ (c_\ell)_3 M \\ (c_\ell)_4 M \end{bmatrix}.$$

Esercizio 5. Si implementi una funzione

```
function W = strassen_matmul(U, V)
```

che calcoli il prodotto $W = U \cdot V$, di due matrici applicando l'algoritmo di Strassen in modo ricorsivo:

- Nel caso in cui U e/o V siano vettori ($m = 1$, o $n = 1$, o $p = 1$), il prodotto viene calcolato con il metodo classico, $W=U*V$.
- Nel caso $m = n = p = 2$ si usa strassen con entrate scalari (si può usare la funzione `matmul_cp` implementata precedentemente).
- Altrimenti si partizionano le matrici come descritto sopra e si applica Strassen con entrate non scalari, chiamando la procedura ricorsivamente quanto si valutano le moltiplicazioni attive.

Si confrontino (su un grafico in scala logaritmica) i tempi di esecuzione di `strassen_matmul` con quelli del metodo classico per matrici $n \times n$, con $n = 2^j$, al crescere di j . Per quale dimensione avviene il sorpasso (Strassen più efficiente di righe $\times$ colonne)?.

Se si vuole rendere il confronto puramente sul numero di operazioni, si può disattivare l'uso di processori in parallelo tramite il comando `maxNumCompThreads`.

Si riporti su un altro grafico l'andamento dell'errore relativo in norma Frobenius del prodotto eseguito con il metodo di Strassen prendendo come quantità “vera” quella calcolata con il metodo classico.

Prodotto di matrici approssimato

Fattorizzazioni CP approssimate (ovvero che evidenziano il border rank) di tensori che rappresentano il prodotto di matrici, possono essere usate per calcolare prodotti di matrici arbitrariamente precisi (almeno in aritmetica esatta). Ad esempio la fattorizzazione

$$\mathbf{A}(\varepsilon) = \begin{bmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \varepsilon & \varepsilon \\ 0 & 0 & 0 & \varepsilon & \varepsilon & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 0 \end{bmatrix},$$

$$\mathbf{B}(\varepsilon) = \begin{bmatrix} \varepsilon & 0 & 0 & -\varepsilon & 0 & 1 & 1 & -1 & 1 & 0 \\ 0 & -1 & 0 & 1 & 0 & 0 & 0 & 0 & \varepsilon & 0 \\ 0 & 0 & 0 & 0 & \varepsilon & 0 & 0 & -1 & 0 & 1 \\ 1 & -1 & 1 & 0 & 1 & \varepsilon & 0 & 0 & 0 & -\varepsilon \end{bmatrix},$$

$$\mathbf{C}(\varepsilon) = \begin{bmatrix} 1/\varepsilon & 1/\varepsilon & -1/\varepsilon & 1/\varepsilon & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & -1/\varepsilon & 0 & 1/\varepsilon & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & -1 & 0 \\ 1 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & -1/\varepsilon & 0 & 1/\varepsilon & 0 \\ 0 & 0 & 0 & 0 & 0 & 1/\varepsilon & -1/\varepsilon & 1/\varepsilon & 0 & 1/\varepsilon \end{bmatrix},$$

è tale per cui

$$\lim_{\varepsilon \rightarrow 0} \llbracket A(\varepsilon), B(\varepsilon), C(\varepsilon) \rrbracket = \langle 3, 2, 2 \rangle.$$

Esercizio 6. Si scriva una funzione

```
function W = app_322_matmul(U, V, t)
```

che calcola il prodotto UV usando la rappresentazione approssimata riportata sopra con $\varepsilon = t$. Suggerimento: si cerchi di riutilizzare la funzione `matmul_cp` implementata prima. Per testare quando si può spingere sull'accuratezza, si generino casualmente $U \in \mathbb{R}^{3 \times 2}$ e $V \in \mathbb{R}^{2 \times 2}$ e si calcoli il prodotto per $t \in \{10^{-1}, 10^{-2}, \dots, 10^{-16}\}$. Infine si riportino su un grafico, in scala logaritmica, l'andamento dell'errore relativo (rispetto al prodotto calcolato col metodo classico) in funzione di t .